Business Requirements Document (BRD)

Project Title: Collaborative Organizational Resource Engine (CORE)
Technology Stack: Java Spring Boot | MySQL | JWT | Spring Security | Hibernate | Redis | AWS-ready **Architecture:** Multi-Tenant SaaS (Shared Database, Organization-based Isolation) **Author:** Vasudev D. **Version:** 1.0
Date: October 2025

---

1. Project Overview

The **Collaborative Organizational Resource Engine** is a comprehensive, modular SaaS platform designed for **organizations of any size** — from startups to large enterprises — to manage their **projects, teams, clients, and operations** efficiently in a single, secure environment.

It enables:

- Centralized employee, team, and client management
- Organization-specific data isolation
- Dynamic role, permission, and policy-driven access control
- Full project lifecycle management with task tracking, bug tracking, and time management
- Secure communication, collaboration, and document sharing
- AI-powered analytics and automation capabilities

---

2. Project Goals

Goal	Description
Multi-Organization Isolation	Ensure that all data created under one organization is fully isolated from others, with no shared visibility.
Dynamic Access Management	Roles, permissions, and resource access must be configurable dynamically — no hardcoding in controllers or code.
Complete HR & Team Management	Manage employees, departments, teams, onboarding, and roles within each organization.

Goal	Description
Project Lifecycle Management	Allow creation, tracking, and delivery of projects across clients and teams.
Client Collaboration Portal	Provide clients with limited access to their projects, tasks, and communications.
Integrated Modules	Unified modules for projects, tasks, bugs, meetings, files, communication, and dashboards.
⚡ Scalable SaaS Foundation	Design the system so it can later evolve into microservices with minimal refactoring.
AI Integration Ready	Support AI-based insights, reporting, and automation in later phases.

---

3. Key Modules & Features

3.1. Organization Management

- Manage multiple organizations using a shared platform.
- Each organization has isolated data space (`organization_id`).
- Configurable settings: working hours, company policies, holidays, etc.

Entities:

- Organization
- OrganizationSetting

3.2. Authentication & Authorization

- Secure JWT-based authentication.
- Dynamic roles, permissions, and policy-driven authorization.
- Tenant-aware login: users belong to specific organizations.
- Admin can define resources (`projects`, `tasks`, `files`, etc.) and actions (`read`, `create`, `update`, `delete`).
- Policies determine who can do what on which resource.

Entities:

- User
- Role
- Permission
- Resource
- Action
- Policy
- Token (JWT refresh handling)

3.3. Employee & Team Management

- Manage employee details, onboarding, deboarding, and employment history.
- Organize employees into departments and teams with defined managers, leads, and tech stacks.
- Track availability, skillset, and working hours.

Entities:

- Employee
- Department
- Team
- TeamMember
- Designation
- EmployeeDocument
- EmploymentHistory

Features:

- Employee lifecycle tracking (joined, promoted, resigned).
- Assign employees to multiple teams.
- Set manager/team lead roles dynamically.
- Team-specific dashboards and analytics.

3.4. Client Management (Full Collaboration)

- Manage client organizations and their users.
- Allow clients to log in and view their associated projects.
- Clients can collaborate on tasks, raise bugs, and communicate securely.

Entities:

- ClientOrganization
- ClientUser
- ClientProjectAccess
- ClientDocument
- ClientCommunicationThread

Features:

- Client onboarding and portal access.
- Client-specific dashboards showing projects, progress, and communications.
- Internal ↔ Client communication threads.
- File sharing with access control.

3.5. Project Management

- Manage projects across multiple teams and clients.
- Track project lifecycle, deliverables, and progress.
- Manage phases, sprints, and documentation.

Entities:

- Project
- ProjectTeam
- ProjectPhase
- ProjectStatus
- ProjectFile

Features:

- Link projects to clients and teams.
- Track milestones and deliverables.
- Maintain project-based communication and file storage.
- Kanban board integration for project status visualization.

3.6. Task Management & Time Tracking

- Create and assign tasks to employees or teams.
- Track progress, deadlines, and time logs.
- Enable team collaboration via comments and attachments.

Entities:

- Task
- TaskAssignment
- TaskComment
- TaskAttachment
- WorkLog
- TaskStatus

Features:

- Role-based task visibility.
- Real-time updates and time tracking.
- Task-based reporting and metrics.

3.7. Bug Tracking

- Centralized bug reporting and management system linked with projects and tasks.

Entities:

- BugReport
- BugPriority
- BugStatus
- BugAttachment
- BugComment

Features:

- Clients and employees can raise bugs.
- Automatic notifications and updates.
- Visual tracking in Kanban boards.

3.8. File Management

- Upload, share, and manage files for teams, projects, and clients.
- Secure file access based on organization, project, or team.

Entities:

- FileResource
- FileTag
- FileAccessControl

Features:

- Role-based file permissions (view/download/edit).
- Project-wise or client-wise folder hierarchy.

3.9. Communication & Notifications

- Built-in communication channels between employees and clients.
- Message threads, replies, and attachments.
- In-app and email notifications for important events.

Entities:

- MessageThread
- Message
- Notification

Features:

- Internal chat system per project or team.
- Integration with meetings and tasks.

- Notification system for events, comments, deadlines, etc.

3.10. Meetings & Scheduling

- Schedule internal and client meetings.
- Attach meeting minutes and action points.
- Sync with employee and project calendars.

Entities:

- Meeting
- MeetingParticipant
- MeetingMinutes

3.11. Dashboards & Analytics

- Dynamic dashboards based on user role (Admin, Manager, Developer, Client).
- Visual reports on team productivity, project progress, client satisfaction, etc.
- Kanban boards for projects and tasks.

Features:

- Time utilization charts.
- Task completion metrics.
- Client project health summary.
- Department-level reports.

3.12. AI & Automation (Phase 2+)

- AI-based project risk prediction.
- Sentiment analysis on client communications.
- Automated status report generation.
- Chatbot for quick data queries (e.g., “Show my pending tasks”).

---

4. Dynamic Role, Permission, and Policy Management

Key Concept: No access control will be hardcoded in the application. All permissions, resources, and policies are dynamically stored in DB and can be configured by Admin users.

Example:

Role	Resource	Action	Condition	Description
Team Lead	Task	Update	task.assignedTo == currentUser	Update own tasks only
Client User	Project	View	project.clientId == currentClient	View only their projects
Admin	*	*	*	Full system access

Implementation Approach:

- Spring AOP for method-level authorization.
- SpEL (Spring Expression Language) for conditional checks.
- Policy caching in Redis for fast evaluation.

---

5. Security & Data Isolation

Security Feature	Description
JWT-based Authentication	Stateless session handling with refresh tokens
Multi-Tenant Isolation	All data filtered by `organization_id` automatically
Hibernate Filters	Applied globally to every entity
Row-level Authorization	Policy-driven access with resource context
Audit Logs	Every action logged (who, what, when, where)
Encryption	Passwords via BCrypt, optional AES for sensitive fields
File Security	Signed URLs for downloads, role-based access

---

6. Non-Functional Requirements

Category	Description
Performance	Must handle 10,000+ concurrent users per instance
Scalability	Horizontal scaling with load balancers
Security	Fully isolated org data, encrypted storage
Availability	99.9% uptime target
Maintainability	Modular code structure, clear domain separation
Extensibility	Future modules (Finance, Payroll, CRM) can be plugged in
Compliance	GDPR & SOC2 readiness

---

7. High-Level Architecture

- **Frontend:** React.js / Next.js (later phase)
- **Backend:** Spring Boot (Modular Monolith, later migratable to Microservices)
- **Database:** MySQL (shared, org_id isolation)
- **Cache:** Redis
- **Storage:** AWS S3 (for file uploads)
- **Auth:** JWT + Refresh Tokens
- **AI Engine:** (Future) OpenAI / Local ML Model Integration

---

8. Phase-wise Implementation Plan

Phase	Module	Duration (Days)
1	Core Setup (Spring Boot, DB, Base Entities)	2

Phase	Module	Duration (Days)
2	Auth & Org Management	4
3	Employee & Team Management	4
4	Project Management	4
5	Client Management	4
6	Task & Bug Tracking	5
7	File Management	3
8	Communication & Meetings	3
9	Dashboards	3
10	AI Integrations & Policy Engine	6

---

9. Deliverables

- Complete backend REST API with Swagger documentation.
- Database schema (MySQL) with all relationships.
- Role-based access and dynamic permission setup.
- Organization-level isolation verified via tests.
- Dockerized deployment ready for AWS.

⚡ 10. Future Enhancements

- Payroll and Expense Tracking module.
- Performance and Appraisal Management.
- Advanced Reporting with PowerBI/Metabase integration.
- AI Copilot for automatic task updates.
- Multi-database isolation for Enterprise Tenants.